

devin × **stripe**

Accelerating SDK Quality with Autonomous Engineering

A proposal for improving test coverage, resolving open issues, and
reducing engineering overhead across Stripe's Java SDK

Presented by Jason Kelley • Prepared for Stripe Engineering Leadership • April 2026 • Confidential

THE OPPORTUNITY

stripe-java powers millions of businesses. Its infrastructure code deserves deeper coverage.

Bug • Open

ClassCastException shadows API errors

Issue #1846 — When the API returns a non-JSON error, users get an uninformative ClassCastException instead of a proper StripeException. Open since August 2024.

Bug • Open PR

NumberFormatException in webhook parsing

PR #2149 — Malformed webhook timestamps cause unhandled exceptions. Community PR submitted January 2026, still open.

Feature • Requested

Micrometer / OpenTelemetry instrumentation

Issue #1964 — Enterprise users need observability hooks for API calls. Workaround requires wrapping the entire library.

Feature • 5 upvotes

GraalVM Native Image support

Issue #1905 — Increasingly critical for Quarkus, Micronaut, and Spring Native users. No official reachability metadata.

THE CHALLENGE

SDK engineers are stretched between generated code and infrastructure quality

stripe-java's codebase is primarily auto-generated from Stripe's OpenAPI spec. The hand-written infrastructure layer — networking, serialization, error handling, webhook verification, retry logic — is where real bugs originate.

With SDK engineers focused on API version releases and code generation, infrastructure improvements accumulate in the backlog. Some issues have been open for 18+ months.

24 open issues

Including bugs that affect production error handling and feature requests with significant community demand.

~30 hand-written files

Core infrastructure code (networking, GSON adapters, webhooks, telemetry) that would benefit from deeper unit test coverage.

THE SOLUTION

Devin: the autonomous software engineer

Devin is an AI software engineer built by Cognition. Unlike copilot-style tools that assist while you code, Devin works independently — exploring codebases, writing code, running tests, iterating on CI, and delivering pull requests.

1

Explore

Autonomously reads source code, tests, and docs to understand conventions

2

Plan

Identifies coverage gaps, prioritizes by risk, and outlines approach

3

Implement

Writes code following existing patterns, imports, and style conventions

4

Validate

Runs lint, tests, and builds. Iterates on failures autonomously

5

Deliver

Opens a PR with clean commits, descriptions, and passing CI

Why Devin, not Cursor or Claude Code?

CAPABILITY	CURSOR / CLAUDE CODE	DEVIN	IMPACT
Autonomy	Human-driven, file-at-a-time	Fully autonomous sessions	Engineer reviews output, not every step
Codebase exploration	Manual file selection	LSP, grep, cross-file analysis	Discovers coverage gaps without guidance
Build & CI	Requires human to run/check	Runs builds, reads CI logs, iterates	Eliminates push-fail-fix-push cycle
Multi-file refactors	Context window limited	Persistent context, unlimited scope	Handles 20-30 files per session
Async execution	Requires engineer in-seat	Runs in background, delivers PR	Zero engineer wait time
Pattern consistency	Varies across prompts	Learns once, replicates uniformly	Consistent test quality across all files

PROPOSED SCOPE

Three workstreams, 3-5 focused PRs

1. Unit Test Coverage

Audit hand-written infrastructure code.
Add edge-case tests for:

- Error handling & parsing
- GSON serialization round-trips
- Webhook signature verification
- Retry logic & rate limiting
- HTTP client behavior
- Telemetry collection

~30 files

2-3 PRs

2. Bug Fixes

Resolve long-standing open issues in
hand-written code:

- #1846 — ClassCastException fix
- #2149 — NumberFormatException fix
- #1925 — Serialization round-trip
- #2001 — API version constants
- #2012 — Builder null handling

5 issues

1-2 PRs

3. Developer Experience

Lay groundwork for high-demand
features:

- JSpecify nullability annotations
- GraalVM reachability metadata
- Improved Javadoc coverage
- Test helper utilities for SDK consumers

Incremental

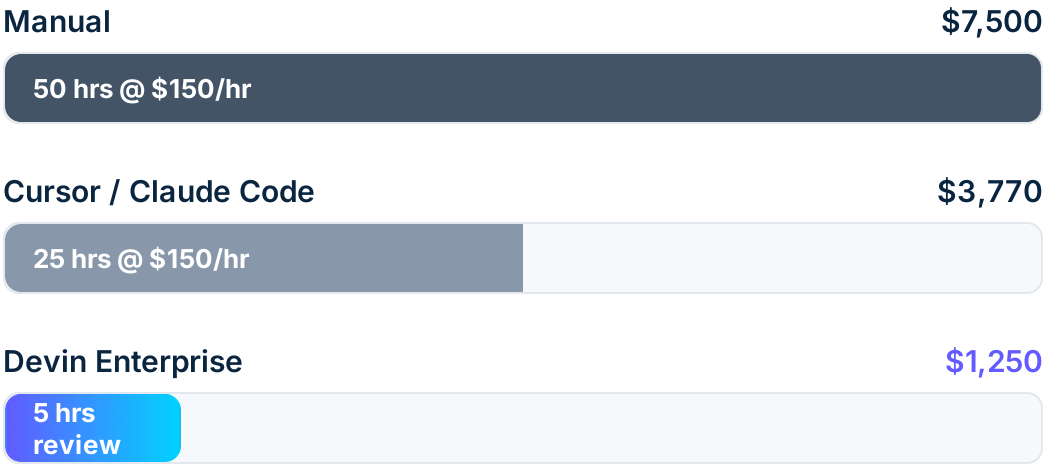
1-2 PRs

COST ANALYSIS

83% cost reduction vs. manual engineering

APPROACH	HUMAN HRS	AI COST	TOTAL
Manual	50 hrs	\$0	\$7,500
Cursor / Claude Code	25 hrs	~\$20	\$3,770
Devin Enterprise	5 hrs	~\$500	\$1,250

Based on \$150/hr fully loaded senior Java engineer cost at Stripe. Devin Enterprise at \$2/ACU.



Net savings: \$6,250 per engagement (83%)

ANNUAL ROI

\$702K–\$1.5M in annual value across Stripe's SDK portfolio

The real savings aren't one-time — they compound monthly as Stripe ships new API versions, each requiring test updates, regression testing, and maintenance across 7+ SDKs and 12+ ecosystem repos.

\$702K

Conservative annual value
Engineering time + recaptured capacity

\$1.5M

Aggressive annual value
Including incident prevention

~14x

Return on Devin spend
\$50K–\$100K/yr Enterprise cost

Real-world precedent: Nubank

Nubank used Devin to migrate their core ETL monolith — 6M+ lines of code. Engineers achieved a **12x efficiency improvement** in engineering hours and **20x cost savings**. Migrations that were projected to take months completed in weeks.

Where the value comes from

Three layers of savings — direct engineering time, recaptured senior capacity, and reduced incident risk — each independently justify the investment.

VALUE DRIVER	HOW	CONSERVATIVE	AGGRESSIVE
Engineering time saved	~20 hrs/mo saved per SDK × 7 SDKs × \$150/hr	\$252K/yr	\$504K/yr
Senior capacity recaptured	5-10 SDK engineers × 20% time redirected to features × \$350-500K fully loaded	\$350K/yr	\$500K/yr
Incident risk reduction	Bugs like #1846 (18mo open) — support tickets, trust erosion, fire drills	\$100K/yr	\$500K/yr
Total annual value		\$702K	\$1.5M
Devin annual cost	Enterprise plan + ACU compute for ongoing SDK maintenance	~\$50K/yr	~\$100K/yr

CROSS-SDK MULTIPLIER

12-15 repos — 7 SDKs + stripe-android, stripe-ios, stripe-react-native, stripe-terminal, and more

RECURRING, NOT ONE-TIME

Monthly API releases — each version requires test updates and regression testing across all SDKs

Built for enterprise security standards

Devin is designed for organizations with the highest security requirements. Every session runs in an isolated environment with full audit trails.



Isolated Execution

Each Devin session runs in its own sandboxed VM. No shared state between sessions. Code never leaves your security perimeter.



Full Audit Trail

Every action — file read, command executed, PR created — is logged and reviewable. Complete transparency for security teams.



Secret Management

Credentials are stored encrypted and injected at runtime. Never written to disk, never exposed in logs or commits.



SOC 2 Type II

Cognition maintains SOC 2 Type II compliance. Enterprise SSO, RBAC, and data retention policies available.



No Training on Your Code

Devin does not train on customer code. Your proprietary logic stays yours. Zero data retention after session ends.



SCM Integration

Native GitHub integration with branch protection, required reviews, and CI gates. Devin never pushes to main.

ENGAGEMENT TIMELINE

From kickoff to merged PRs in 2 weeks

WEEK 1 Discovery & First PRs • Clone repo, analyze codebase structure • Identify test coverage gaps • Submit bug fix PRs (#1846, #2149, #2001) • Begin unit test coverage PRs	WEEK 2 Coverage & Polish • Complete test coverage across 30 files • Iterate on PR review feedback • Begin JSpecify / GraalVM groundwork • Deliver final coverage report	ONGOING Continuous Support • Monitor new issues & regressions • Extend to other Stripe SDKs • Maintain test coverage as API evolves • Scale to broader infrastructure work
Deliverables: 2-3 bug fix PRs, coverage gap report	Deliverables: 2-3 test coverage PRs, DX improvement PR	Deliverables: Ongoing PRs, multi-SDK coverage

Let's ship better SDKs, faster.

Start with stripe-java. Prove the model. Scale across the portfolio.
Your engineers focus on what matters — we handle the rest.

[Start a Pilot](#)

[Schedule a Demo](#)

[devin.ai](#) • hello@cognition.ai

APPENDIX

Detailed Issue Breakdown

ISSUE	TYPE	EFFORT	STATUS	DEVIN APPROACH
#1846 — ClassCastException	Bug	~2 hrs	Open 18 mo	Add catch clause + tests
#2149 — NumberFormatException	Bug	~2 hrs	Open PR	Improve existing PR + tests
#2001 — Version constants	Feature	~1 hr	Approved by Stripe	Add constants to Stripe.java
#1925 — GSON round-trip	Bug	~4 hrs	Open	Verify #2168 fix, add regression tests
#2024 — JSpecify nullability	Feature	~8 hrs	Open	Annotate public API incrementally
#1964 — OpenTelemetry	Feature	~12 hrs	Open	Add listener/hook API, examples
#1905 — GraalVM	Feature	~16 hrs	Open, 5 upvotes	Generate reachability metadata